

Gradient Boosting — Detaylı Konu Anlatımı

Sınav Çalışma Notları

Contents

1. Gradient Boosting’i “Hata Düzeltme” Olarak Düşünmek	1
2. Basit Bir Örnekle Artıklara Ağaç Kurma (Regresyon)	2
3. Öğrenme Oranı (Learning Rate / Shrinkage)	2
4. Genel Çerçeve: Kayıp Fonksiyonu ve Pseudo-Residual	2
5. İkili Sınıflandırma için Gradient Boosting	3
6. GradientBoostingScratch Sınıfı — Fit Sürecinin Mantığı	3
7. Eğitim Kaybının İterasyonlar Boyunca Azalması	3
8. $n_estimators$, $learning_rate$ ve max_depth Etkileşimi	4
9. XGBoost’a Geçiş: Gradient Boosting Neyi Eksik Bırakıyor?	4
10. Hiperparametreler — Sınav İçin Özet Tablo	4
11. Gradient Boosting’in Diğer Yöntemlerle Karşılaştırması	5
12. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma	5

1. Gradient Boosting’i “Hata Düzeltme” Olarak Düşünmek

En basit haliyle Gradient Boosting’i “**hatayı adım adım kapatan bir dizi ağaç**” olarak düşünebiliriz. AdaBoost’tan farklı olarak, Gradient Boosting **örnek ağırlıklarını** değil, doğru-
dan **artıkları (residuals)** hedef alır.

Genel Fikir:

1. Basit bir başlangıç tahmini yap (örneğin, hedef değişkenin ortalaması).
2. Bu tahminin ne kadar yanlış olduğunu (**artık = gerçek – tahmin**) hesapla.
3. Bu artıkları tahmin etmeye çalışan yeni bir ağaç kur.
4. Bu ağacın tahminini (küçültülmüş bir katsayıyla) mevcut modele **ekle**.
5. 2-4 adımlarını $n_estimators$ kez tekrarla.

Her yeni ağaç, bir öncekinin bıraktığı hatayı biraz daha kapatmaya çalışır — tıpkı bir hedefe adım adım yaklaşmak gibi.

2. Basit Bir Örnekle Artıklara Ağaç Kurma (Regresyon)

Küçük bir regresyon veri seti üzerinde süreç şöyle işler:

1. **Başlangıç tahmini:** $F_0(x) = \text{ortalama}(y)$ — yani her örnek için aynı, sabit bir tahmin.
2. **Artıklar hesaplanır:** $r_i = y_i - F_0(x_i)$.
3. Bu artıkları hedef alan bir regresyon ağacı $h_1(x)$ kurulur (ağaç, “hangi bölgede ne kadar hata var” örüntüsünü öğrenir).
4. **Model güncellenir:** $F_1(x) = F_0(x) + \text{learning_rate} * h_1(x)$.
5. Yeni artıklar $r_i = y_i - F_1(x_i)$ hesaplanır ve süreç tekrarlanır.

Her iterasyonda toplam hata (artıkların büyüklüğü) küçülmeye devam eder — bu, algoritmanın **kayıp fonksiyonunu doğrudan minimize eden bir gradyan inişi (gradient descent)** süreci olduğunun sezgisel açıklamasıdır.

3. Öğrenme Oranı (Learning Rate / Shrinkage)

Yukarıdaki örnekte her ağacın düzeltilmesi **tam olarak** uygulanmaz; bir `learning_rate` (küçültme/shrinkage) katsayısıyla ölçeklenir:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x), \quad 0 < \eta \leq 1$$

Neden Tam Uygulanmaz? Eğer her ağacın düzeltilmesi tam uygulansaydı, model eğitim veri-sine çok hızlı ve aşırı uyum sağlardı (overfitting). Küçük adımlarla ilerlemek (η küçük), modelin daha **yumuşak ve genelleyci** bir yol izlemesini sağlar.

Bias-Variance-Computation Üçgeni:

- **Düşük learning_rate** (örn. 0.01): Yavaş öğrenir, daha fazla ağaç ($n_{\text{estimators}}$) gerektirir, ama genelde **daha iyi genelleme** sağlar.
- **Yüksek learning_rate** (örn. 0.5): Hızlı öğrenir, az ağaçla yüksek eğitim performansına ulaşır, ama **overfitting riski** yüksektir.
- Pratikte düşük `learning_rate` + yüksek $n_{\text{estimators}}$ kombinasyonu tercih edilir (hesaplama maliyeti artsa da).

4. Genel Çerçeve: Kayıp Fonksiyonu ve Pseudo-Residual

Regresyon örneğimizde “artık” dediğimiz şey, aslında **kare hata kaybının (squared error loss) negatif gradyanıdır**:

$$L(y, F(x)) = \frac{1}{2}(y - F(x))^2 \Rightarrow -\frac{\partial L}{\partial F(x)} = y - F(x)$$

Bu genelleme sayesinde Gradient Boosting, **herhangi bir türevlenebilir kayıp fonksiyonuyla** çalışabilir:

- Regresyon → kare hata (MSE) veya mutlak hata (MAE) kaybı,
- İkili sınıflandırma → log-loss (log-likelihood) kaybı,
- Sıralama (ranking) → özel sıralama kayıpları.

Her durumda algoritma aynı kalır: **kayıp fonksiyonunun negatif gradyanını (pseudo-residual) tahmin eden bir ağaç kur, modele ekle, tekrarla.** Bu yüzden yönteme “Gradient” (gradyan) Boosting denir.

5. İkili Sınıflandırma için Gradient Boosting

Sınıflandırmada model, doğrudan sınıf tahmini değil, bir **log-odds (logit)** skoru üretir. Bu skor, olasılığa dönüştürülmek için **sigmoid fonksiyonundan** geçirilir:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sayısal kararlılık için z değeri genelde $[-500, 500]$ aralığına kırılır (`np.clip`) — çünkü çok büyük negatif/pozitif z değerleri `exp()` fonksiyonunda taşmaya (overflow) neden olabilir.

Pseudo-Residual (Sınıflandırmada): Log-loss kaybının negatif gradyanı şu şekildedir:

$$r_i = y_i - \sigma(F(x_i))$$

Yani gerçek etiket (0 veya 1) ile modelin ürettiği olasılık tahmini arasındaki fark, tıpkı regresyondaki gibi bir sonraki ağacın hedefi olur.

6. GradientBoostingScratch Sınıfı — Fit Sürecinin Mantığı

`fit()` metodundaki hazırlık adımları:

1. Başlangıç tahmini F_0 hesaplanır (regresyonda ortalama, sınıflandırmada log-odds tabanlı sabit).
2. `n_estimators` kez şu döngü tekrarlanır:
 - Mevcut $F(x)$ ile pseudo-residual hesaplanır.
 - Pseudo-residual’ı hedef alan sığ bir regresyon ağacı (`max_depth` genelde 2-5) kurulur.
 - Model, bu ağacın `learning_rate` ile ölçeklenmiş tahminiyle güncellenir.
3. Sınıflandırmada final tahmin, $\sigma(F(x)) > 0.5$ eşliğiyle sınıfa dönüştürülür.

Kritik Nokta: Her ağaç, önceki tüm ağaçların **toplu tahminine göre kalan hatayı** öğrenir — bu nedenle ağaçlar birbirinden bağımsız değildir (Random Forest’in aksine) ve **sıralı olarak** kurulmak zorundadır.

7. Eğitim Kaybının İterasyonlar Boyunca Azalması

Eğitim sürecinde kayıp değeri (örn. MSE veya log-loss) **monoton şekilde azalır** — her yeni ağaç, tanım gereği mevcut hatayı azaltacak yönde eklenir. Bu, gradyan inişinin temel özelliğidir: her adım, kayıp fonksiyonunun negatif gradyanı yönünde ilerler.

Ancak Dikkat: Eğitim kaybının sürekli azalması, **test performansının da sürekli iyileştiği anlamına gelmez.** Belirli bir iterasyondan sonra model, eğitim verisindeki gürültüyü ezberlemeye başlayabilir (overfitting) — bu yüzden eğitim ve test kaybı **birlikte** izlenmelidir (bkz. erken durdurma, Bölüm 9).

8. n_estimators, learning_rate ve max_depth Etkileşimi

Bu üç parametre birbiriyle sıkı bağlıdır ve birlikte ayarlanmalıdır:

Senaryo	Etki
Düşük learning_rate + yüksek n_estimators	Yavaş ama sağlam öğrenme, genelde en iyi genelleme
Yüksek learning_rate + düşük n_estimators	Hızlı öğrenme, overfitting riski yüksek
Yüksek max_depth	Her ağaç daha karmaşık örüntüler öğrenir, ama overfitting riski artar (genelde 2-5 arası tercih edilir)
Düşük max_depth (sığ ağaçlar)	Her ağaç zayıf kalır ama boosting süreci bunu telafi eder — Gradient Boosting’de genelde sığ ağaçlar tercih edilir

Pratik Kural: max_depth’i düşük tutup (2-5), learning_rate’i küçük seçip (0.01-0.1), n_estimators’ı artırmak (100-1000) genelde en iyi sonucu verir.

9. XGBoost’a Geçiş: Gradient Boosting Neyi Eksik Bırakıyor?

Sıfırdan yazdığımız (veya scikit-learn’ün GradientBoostingClassifier/Regressor’ı) klasik Gradient Boosting, **temel matematiği** doğru uygular ama şu pratik konularda sınırlı kalır:

- **Regularizasyon eksikliği:** Ağırlıkların/yaprakların büyüklüğüne ceza uygulamaz (XGBoost L1/L2 regularizasyon ekler).
- **Erken durdurma (early stopping) yerleşik değil:** Doğrulama setinde performans kötüleşince otomatik durma mekanizması standart değildir (XGBoost’ta yerleşiktir).
- **Hız/ölçeklenebilirlik:** Büyük veri setlerinde yavaş kalabilir (XGBoost histogram tabanlı hızlandırmalar sunar).
- **Eksik veri yönetimi:** Manuel doldurma gerektirir (XGBoost eksik değerler için otomatik yön öğrenir).

Bu eksiklikler, XGBoost ve LightGBM gibi “gelişmiş” boosting kütüphanelerinin geliştirilme motivasyonunu oluşturur (detaylar için XGBoost ve LightGBM konu anlatımlarına bakınız).

10. Hiperparametreler — Sınav İçin Özet Tablo

Parametre	Anlamı	Etkisi
n_estimators	Ağaç sayısı	Fazlası overfitting riski taşır (AdaBoost gibi)

Parametre	Anlamı	Etkisi
learning_rate	Küçültme (shrinkage) katsayısı	Düşükse daha fazla ağaç gerekir, genelde daha iyi genelleme
max_depth	Her ağacın derinliği	Genelde sığ tutulur (2-5); derin ağaçlar overfitting riski taşır
subsample	Her ağaç için kullanılan satır oranı (Stochastic GB)	<1.0 iken varyansı azaltır, hızlandırır
loss	Kayıp fonksiyonu	Regresyonda squared_error/absolute_error; sınıflandırmada log_loss

11. Gradient Boosting'in Diğer Yöntemlerle Karşılaştırması

Kriter	Random Forest	AdaBoost	Gradient Boosting
Eğitim stratejisi	Paralel	Sıralı	Sıralı
Neyi hedefler	Varyans azaltma	Örnek ağırlıklandırma	Artık/gradyan tahmini
Zayıf öğrenici	Genelde orta-derin ağaç	Karar kütüğü (max_depth=1)	Sığ ağaç (max_depth 2-5)
Kayıp fonksiyonu esnekliği	Yok (doğrudan oylama)	Üstel kayıp (implicit)	Herhangi bir türevlenebilir kayıp
Aykırı değere duyarlılık	Düşük	Yüksek	Orta (kayıp fonksiyonuna göre değişir)

12. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma

- Gradient Boosting, her yeni ağacı **önceki modelin artıklarını (residuals)** ya da genel olarak **kayıp fonksiyonunun negatif gradyanını (pseudo-residual)** tahmin edecek şekilde kurar.
- learning_rate (shrinkage), her ağacın katkısını ölçekler; düşük değer + çok ağaç genelde en iyi genelleme sağlar.
- Sınıflandırmada model **log-odds** üretir, sigmoid ile olasılığa dönüştürülür.
- Eğitim kaybı **monoton azalır** ama bu, test performansının da arttığı anlamına gelmez — overfitting riski vardır.
- Gradient Boosting **sıralı** eğitim yapar → Random Forest gibi **parallelleştirilemez**.
- Genelde **sığ ağaçlar** (max_depth=2-5) tercih edilir — her ağaç “zayıf” kalır, boosting süreci bunları güçlü bir modele dönüştürür.
- XGBoost/LightGBM, Gradient Boosting'in regularizasyon, hız ve ölçeklenebilirlik eklenmiş **gelişmiş versiyonlarıdır**.